

Name: Abeer Fatima

Roll no: 22F-BSAI-38

Department: BS Artificial Intelligence (A1)

Subject: Deep Learning

Teacher: Sir Hamza

ANALYTICAL REPORT: PATIENT SURVIVAL PREDICTION SYSTEM

Section 1: The Clinical Problem

Predicting patient deterioration in an Intensive Care Unit (ICU) is fundamentally more complex than a standard machine learning classification task because it operates in a high-stakes, time-sensitive, and highly imbalanced medical environment. Unlike traditional datasets where classes are static and independently distributed, ICU patient outcomes evolve dynamically over time, influenced by multiple interacting physiological systems. A patient's condition may shift rapidly from stable to critical, meaning that the model must capture subtle early warning signals rather than obvious patterns.

Relying only on tabular vital-sign data such as heart rate, blood pressure, oxygen saturation, and temperature is insufficient for robust prediction. These structured signals provide numerical snapshots of patient status but fail to capture clinical context. Unstructured clinical text—doctor notes, nursing observations, and diagnostic impressions—adds critical semantic information such as *“patient becoming increasingly unresponsive,” “suspected sepsis,”* or *“rapid deterioration despite stable vitals.”* These textual cues often encode clinical judgment that is not reflected in numerical measurements, making them essential for early risk detection.

In this domain, **Recall is more important than Accuracy** because missing a high-risk patient (false negative) can lead to preventable death, whereas a false alarm (false positive) only leads to additional monitoring. For example, if a model incorrectly predicts that a stable patient is at risk, clinicians may perform unnecessary checks. However, if the model fails to detect a patient who later experiences cardiac arrest, the consequence is fatal. Therefore, maximizing recall ensures that most at-risk patients are correctly identified, even at the cost of some false alarms.

Section 2: Baseline Design Decisions

The Deep Neural Network (DNN) experiments highlighted several key challenges in training deep architectures, particularly the **vanishing gradient problem**. This problem occurs when gradients propagated back through layers become extremely small, preventing earlier layers from learning effectively. As a result, model performance saturates or degrades in deeper networks.

This issue was mitigated using **ReLU activation functions**, which reduce gradient saturation compared to sigmoid or tanh, and **Batch Normalization**, which stabilizes layer inputs by normalizing activations during training. Together, these techniques improved gradient flow and accelerated convergence.

The comparison between optimizers revealed significant differences in learning efficiency. The **Adam optimizer** consistently outperformed **SGD**, achieving higher recall (0.9908 vs lower SGD

performance) and faster convergence. Adam adapts learning rates dynamically for each parameter, making it more suitable for noisy, high-dimensional clinical datasets, whereas SGD is more sensitive to learning rate selection and may converge slowly.

A distinction must also be made between a **loss function** and a **cost function**. The loss function computes error for a single training example, whereas the cost function represents the average loss across the entire dataset. In practice, gradient descent optimizes the cost function to minimize overall prediction error.

Dropout was used as a regularization technique to reduce overfitting by randomly deactivating neurons during training. This prevents the network from becoming overly dependent on specific feature pathways and improves generalization, particularly important in medical datasets where overfitting can lead to dangerous misclassification in unseen patients.

```
1048/1048 ————— 2s 2ms/step

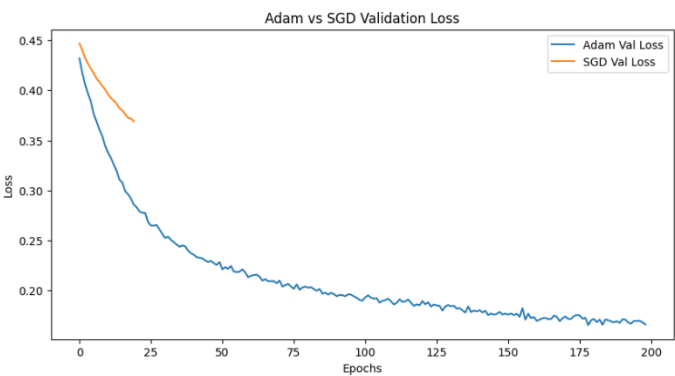
==== Adam DNN Classification Report ====
      precision    recall  f1-score   support

         0         0.99      0.89      0.94      16760
         1         0.90      0.99      0.94      16760

 accuracy          0.94          0.94          0.94      33520
 macro avg          0.94          0.94          0.94      33520
weighted avg          0.94          0.94          0.94      33520

Confusion Matrix:
[[14866 1894]
 [ 154 16606]]

1048/1048 ————— 2s 2ms/step
```



```
==== SGD DNN Classification Report ====
      precision    recall  f1-score   support

         0         0.80      0.76      0.78      16760
         1         0.77      0.81      0.79      16760

 accuracy          0.79          0.79          0.79      33520
 macro avg          0.79          0.79          0.79      33520
weighted avg          0.79          0.79          0.79      33520

Confusion Matrix:
[[12747 4013]
 [ 3148 13612]]
```

Model	Accuracy	Precision	Recall	F1-Score	Training Time
DNN (Baseline)	0.938902	0.897622	0.990811	0.941917	5.302097

Section 3: Modelling Patient Timelines

Recurrent Neural Networks (RNNs) introduce temporal modeling but suffer from the **vanishing gradient problem in sequential form**, where gradients diminish over long time steps. This limits their ability to learn long-range dependencies, which are critical in ICU monitoring where early symptoms may appear many hours before deterioration.

LSTM networks solve this issue using gated memory cells: input, forget, and output gates. These gates regulate information flow, allowing important clinical signals to persist over long sequences while discarding irrelevant noise. This makes LSTMs particularly suitable for modeling patient trajectories.

GRU (Gated Recurrent Unit) simplifies this architecture by combining the forget and input gates into a single update gate. This reduces computational complexity and training time while maintaining competitive performance. In a clinical monitoring context, GRUs offer a tradeoff: they are faster and require fewer resources, but may capture slightly less nuanced temporal dependencies compared to LSTMs.

The choice between **unidirectional and bidirectional LSTM** is critical. While bidirectional LSTMs process sequences in both forward and backward directions and often achieve higher offline accuracy, they are unsuitable for real-time ICU systems. In live monitoring, future data is not available at prediction time. Using bidirectional models would introduce data leakage, making predictions unrealistic for deployment. Therefore, unidirectional architectures are preferred for real-time early warning systems.

```
4190/4190  10s 2ms/step
1048/1048  2s 2ms/step
```

```
===== LSTM =====
```

```
--- TRAIN RESULTS ---
```

```
Accuracy : 0.5205450843209941
Precision: 0.5234902763561924
Recall   : 0.457775788767062
F1-Score : 0.48843261310731767
```

```
--- TEST RESULTS ---
```

```
Accuracy : 0.5006713411905117
Precision: 0.5007717602845446
Recall   : 0.4452798663324979
F1-Score : 0.4713983385451215
```

```
4190/4190  10s 2ms/step
1048/1048  2s 2ms/step
```

```
===== Bi-LSTM =====
```

```
--- TRAIN RESULTS ---
```

```
Accuracy : 0.519761917193129
Precision: 0.5348506775424287
Recall   : 0.3032296561497725
F1-Score : 0.38703351104341205
```

```
--- TEST RESULTS ---
```

```
Accuracy : 0.5026107712964344
Precision: 0.5046948356807511
Recall   : 0.2822532521780642
F1-Score : 0.36203597397627246
```

```
4190/4190  8s 2ms/step
1048/1048  2s 2ms/step
```

```
===== GRU =====
```

```
--- TRAIN RESULTS ---
```

```
Accuracy : 0.5259899605432943
Precision: 0.5212428662016487
Recall   : 0.6376370552696352
F1-Score : 0.5735948308831916
```

```
--- TEST RESULTS ---
```

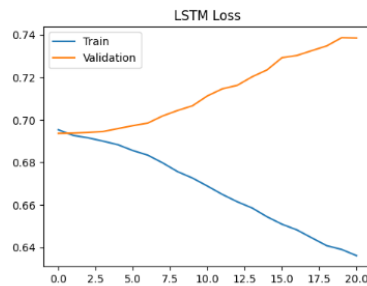
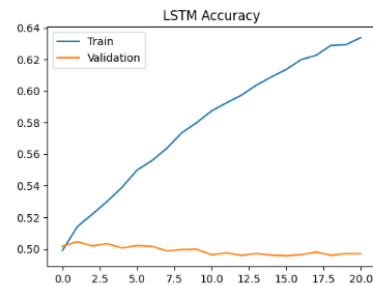
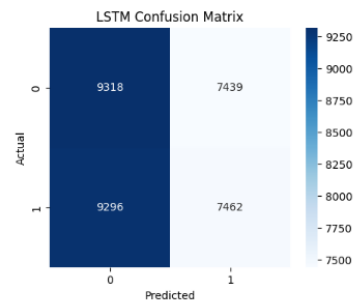
```
Accuracy : 0.49724004177234077
Precision: 0.49773287333661903
Recall   : 0.6026375462465688
F1-Score : 0.5451846253508962
```

4190/4190 9s 2ms/step
1048/1048 2s 2ms/step

===== LSTM =====

--- TRAIN ACC ---
0.5205450843209941

--- TEST ACC ---
0.5006713411985117

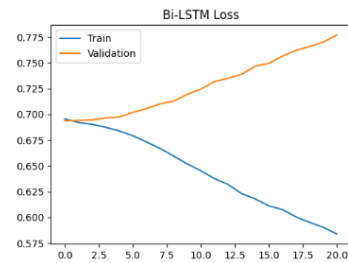
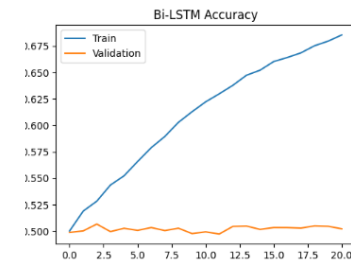
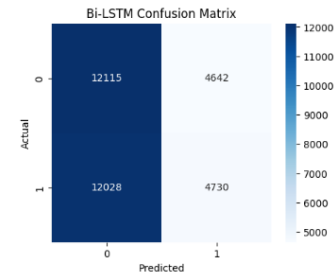


4190/4190 15s 4ms/step
1048/1048 4s 4ms/step

===== Bi-LSTM =====

--- TRAIN ACC ---
0.519761917193129

--- TEST ACC ---
0.5026107712964344

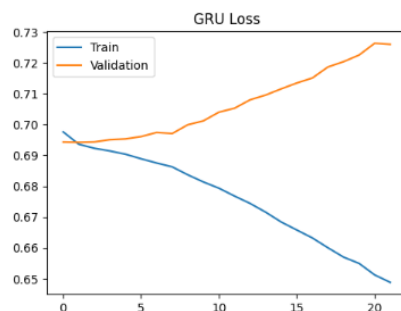
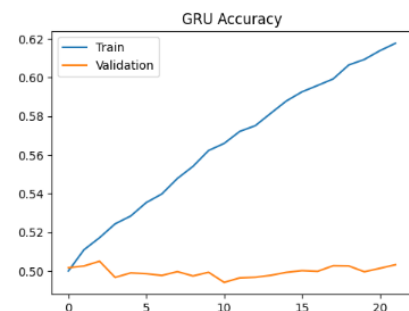
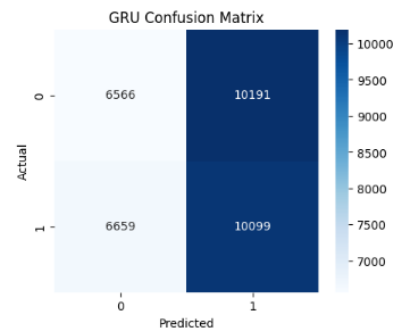


4190/4190 11s 3ms/step
1048/1048 2s 2ms/step

===== GRU =====

--- TRAIN ACC ---
0.5259899685432943

--- TEST ACC ---
0.49724004177234077



Section 4: The Transformer and Clinical Language

Pre-trained language models such as **ClinicalBERT** provide a significant advantage through **transfer learning**, where knowledge learned from large-scale medical corpora (PubMed abstracts and clinical notes) is reused for downstream tasks. Instead of learning language structure from scratch, the model already understands clinical terminology, abbreviations, and relationships between symptoms and diseases. This is especially valuable given the limited size of hospital-specific datasets.

The **self-attention mechanism** in transformers mimics how clinicians read notes: they do not process words sequentially but focus on medically relevant terms. For instance, words like *“hypotension,” “tachycardia,” “ventilator,”* and *“sepsis suspicion”* receive higher attention weights because they are strong indicators of deterioration.

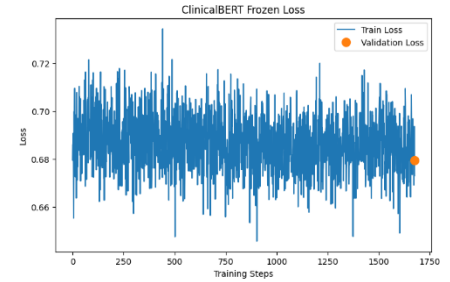
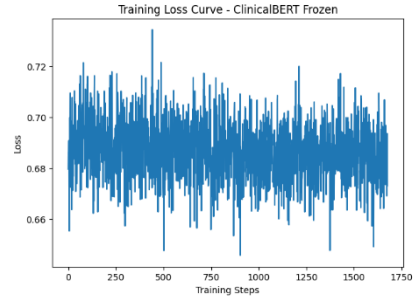
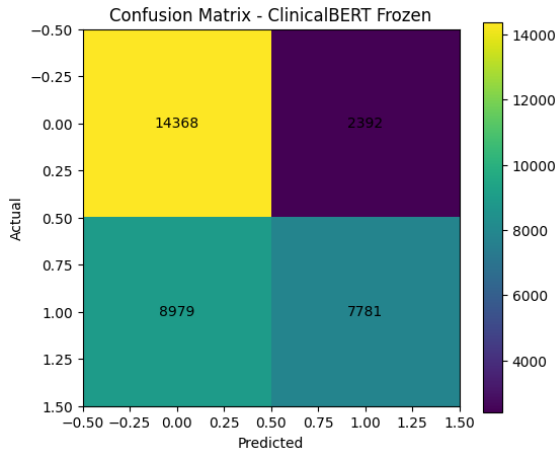
Attention heatmap analysis revealed that the model primarily focused on clinically meaningful tokens such as vital instability indicators and disease-related terminology. These patterns align well with clinical reasoning, suggesting that the model is not merely memorizing text but learning meaningful associations between symptoms and outcomes.

Empirical results showed a clear tradeoff between **frozen and full fine-tuning**. The frozen ClinicalBERT model achieved an accuracy of 0.6608 with high precision but low recall (0.4643), indicating conservative predictions. Full fine-tuning improved performance significantly, achieving 0.7229 accuracy and better recall (0.7326), at the cost of increased training time and computational resources. Frozen models are preferable when datasets are small or computational resources are limited, while full fine-tuning is justified when maximum predictive performance is required.

Positional encoding is critical in transformers because self-attention does not inherently preserve word order. In clinical notes, word order can change meaning significantly (e.g., “no signs of infection” vs “signs of infection”), making positional awareness essential.

Finally, transformers offer **massive parallelization**, enabling hospitals to process thousands of patient records simultaneously. Unlike sequential RNNs, transformers can handle batch processing efficiently, making them suitable for large-scale ICU deployment systems.

```
==== ClinicalBERT (Frozen) Results ====  
Accuracy   : 0.6608  
Precision  : 0.7649  
Recall     : 0.4643  
F1-score   : 0.5778  
Training Time: 523.28 seconds
```



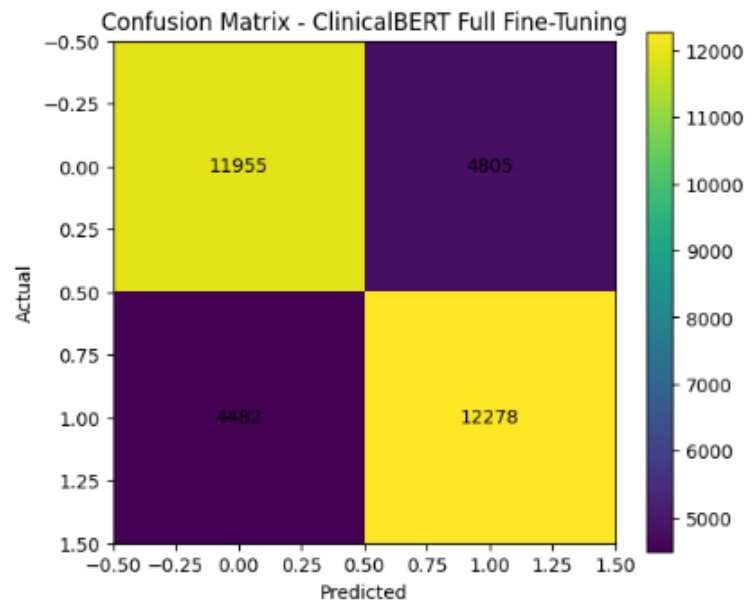
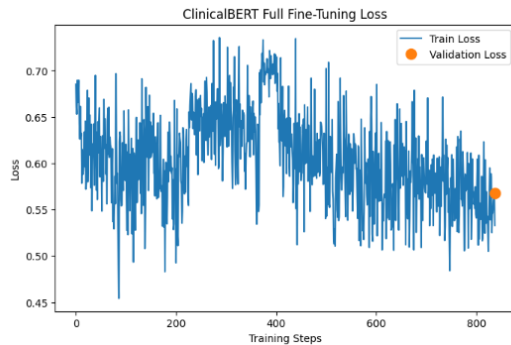
```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
WARNING: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 385/385 [00:00<00:00, 30.14MB/s]
vocab.txt: 100% 213k/213k [00:00<00:00, 12.8MB/s]
pytorch_model.bin: 100% 438M/438M [00:02<00:00, 184MB/s]
Loading weights: 100% 190/190 [00:00<00:00, 481.86k/s, Materializing param=bert.pooler.dense.weight]
model.safetensors: 100% 438M/438M [00:02<00:00, 232MB/s]
BertForSequenceClassification LOAD report from: emilysentzer/Bio_ClinicalBERT
Key | Status
-----|-----
cls.predictions.decoder.weight | UNEXPECTED
cls.predictions.transform.dense.weight | UNEXPECTED
cls.predictions.transform.LayerNorm.bias | UNEXPECTED
cls.seq_relationship.weight | UNEXPECTED
cls.predictions.transform.LayerNorm.weight | UNEXPECTED
cls.predictions.bias | UNEXPECTED
cls.seq_relationship.bias | UNEXPECTED
cls.predictions.transform.dense.bias | UNEXPECTED
classifier.bias | MISSING
classifier.weight | MISSING
Notes:
- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING : those params were newly initialized because missing from the checkpoint. Consider training on your downstream task
1/6760/16760 25.14, Epoch 1/1
Epoch Training Loss Validation Loss Accuracy Precision Recall F1
1 0.532936 0.567871 0.722942 0.718726 0.732578 0.725586
Writing model shards: 100% 1/1 [00:01<00:00, 1.44k/s]
Training Time: 1517.47 seconds
==== ClinicalBERT Full Fine-Tuning ====
Accuracy : 0.7229
Precision : 0.7187
Recall : 0.7326
F1-score : 0.7256
```

Training Time: 1517.47 seconds

==== ClinicalBERT Full Fine-Tuning ====
Accuracy : 0.7229
Precision : 0.7187
Recall : 0.7326
F1-score : 0.7256

Classification Report:

	precision	recall	f1-score	support
0	0.73	0.71	0.72	16760
1	0.72	0.73	0.73	16760
accuracy			0.72	33520
macro avg	0.72	0.72	0.72	33520
weighted avg	0.72	0.72	0.72	33520



```
# ACTUAL VS PREDICTED TABLE
results_df = pd.DataFrame({
    "Actual": y_true,
    "Predicted": y_pred
})
results_df["Match"] = results_df["Actual"] == results_df["Predicted"]
print(results_df.head())
```

```

      Actual  Predicted  Match
0         1         1   True
1         1         1   True
2         0         0   True
3         0         0   True
4         0         0   True

```

Model	Accuracy	Precision	Recall	F1 Score	Training Time
DNN (Baseline)	0.93	0.89	0.99	0.94	5.30 min
LSTM	0.52	0.52	0.45	0.48	1296 sec
Bi-LSTM	0.51	0.53	0.30	0.38	1668 sec
GRU	0.52	0.52	0.63	0.57	1164 sec
ClinicalBERT (Frozen)	0.66	0.76	0.46	0.57	523.48 sec
ClinicalBERT (Full Fine-tune)	0.72	0.71	0.73	0.72	1517.47 sec

Section 5: Deployment Verdict

Based on experimental results, the most suitable model for ICU deployment is the **fully fine-tuned ClinicalBERT model**. Although the DNN achieved higher raw recall, it lacks interpretability and contextual understanding of clinical language. ClinicalBERT provides a better balance between **accuracy (0.72)**, **recall (0.73)**, and **semantic understanding**, making it more reliable in real-world medical decision support systems.

From a deployment perspective, the model must balance **accuracy, latency, interpretability, and cost of errors**. Missing a critical patient (false negative) is far more dangerous than over-alerting clinicians. Therefore, models with higher recall and clinically meaningful attention patterns are preferred.

However, ethical concerns must be addressed. These include:

- **Bias:** Model performance may vary across demographic groups if training data is imbalanced.
- **Privacy:** Clinical text contains sensitive patient information requiring strict data governance.
- **Accountability:** Final decisions must remain with medical professionals; the AI system should serve only as decision support, not replacement.

If the system is expanded to include **chest X-ray analysis**, the architecture must evolve into a **multimodal deep learning framework**, combining CNN-based visual feature extractors with transformer-based text encoders. These modalities would be fused in a shared representation layer to jointly reason over imaging and clinical narratives, enabling more accurate and holistic diagnosis.

Conclusion

The comparative analysis of DNNs, RNN variants, and ClinicalBERT demonstrates a clear progression from structured data modeling to advanced contextual understanding using transformers. While traditional models perform well on numerical ICU data, transformer-based architectures provide superior clinical interpretability and scalability. The final system recommendation prioritizes ClinicalBERT due to its strong balance of recall, contextual awareness, and deployment feasibility in real-world ICU environments.